

SQL - Funkcje okna (Window functions) Lab 2

Imiona i nazwiska: Mateusz Lampert, Marek Małek

Celem ćwiczenia jest zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu “tradycyjnych” konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022
- PostgreSQL - wersja 15/16/17
- SQLite
- Narzędzia do komunikacji z bazą danych
 - SSMS - Microsoft SQL Managment Studio
 - DtataGrip lub DBeaver
- Przykładowa baza Northwind/Northwind3
 - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019
- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020
- Kilka linków do materiałów które mogą być pomocne - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16>
 - <https://www.sqlservertutorial.net/sql-server-window-functions/>
 - <https://www.sqlshack.com/use-window-functions-sql-server/>
 - <https://www.postgresql.org/docs/current/tutorial-window.html>
 - <https://www.postgresqltutorial.com/postgresql-window-function/>
 - <https://www.sqlite.org/windowfunctions.html>
 - <https://www.sqlitetutorial.net/sqlite-window-functions/>
- W razie potrzeby - opis Ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
 - <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
 - na lokalnym serwerze MS SQL
 - na lokalnym serwerze PostgreSQL
 - z lokalną bazą SQLite

Można też skorzystać z innych narzędzi klienckich (wg własnego uznania)

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Korzystamy ze “zmodyfikowanej wersji” bazy northwind

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Zadanie 1

Funkcje rankingu, row_number(), rank(), dense_rank()

```
select productid, productname, unitprice, categoryid,
       row_number() over(partition by categoryid order by unitprice desc) as rowno,
       rank() over(partition by categoryid order by unitprice desc) as rankprice,
       dense_rank() over(partition by categoryid order by unitprice desc) as denserankprice
from products;
```

Wykonaj polecenie, zaobserwuj wynik. Porównaj funkcje row_number(), rank(), dense_rank(). Skomentuj wyniki.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite.

Wyniki:

Analiza została przeprowadzona z wykorzystaniem bazy danych PostgreSQL.

Wynik zapytania z polecenia:

	productid	productname	unitprice	categoryid	rowno	rankprice	denserankprice
1	38	Côte de Blaye	263.5	1	1	1	1
2	43	Ipoh Coffee	46	1	2	2	2
3	2	Chang	19	1	3	3	3
4	1	Chai	18	1	4	4	4
5	76	LakkaIikööri	18	1	5	4	4
6	35	Steeleye Stout	18	1	6	4	4
7	39	Chartreuse verte	18	1	7	4	4
8	70	Outback Lager	15	1	8	8	5
9	34	Sasquatch Ale	14	1	9	9	6
10	67	Laughing Lumberjack Lager	14	1	10	9	6
11	75	Rhönbräu Klosterbier	7.75	1	11	11	7
12	24	Guaraná Fantástica	4.5	1	12	12	8
13	63	Vegie-spread	43.9	2	1	1	1

Wizualizacja 1: Zadanie 1 - wynik zapytania rankingowego (PostgreSQL)

Komentarz:

- row_number to numer danego wiersza wewnątrz okna, zgodnie z zadaną kolejnością (tutaj: unitprice desc, natomiast zgodnie z dokumentacją oracle, funkcja row_number nie jest deterministyczna w przypadku remisów). W przypadku row_number nie ma remisów w wartościach, zawsze dostajemy wartości 1..<ilosc wierszy w oknie>.
- rank to ranga danego wiersza wewnątrz okna, zgodnie z zadaną kolejnością (tutaj: unitprice desc). W przypadku remisów, wszystkie wiersze o tej samej wartości dostają tę samą rangę r, natomiast wiersze o następnej w kolejności wartości dostają rangę r+k, gdzie k-ilość remisujących wierszy. rank zawsze przyjmowało wartości 1..<ilosc wierszy w oknie>.
- dense_rank działa podobnie jak rank, natomiast w przypadku remisów, wszystkie wiersze o tej samej wartości dostają tę samą rangę r, a wiersze o następnej w kolejności wartości dostają rangę r+1 (nie przeskakujemy wartości).

Te same wyniki można także uzyskać nie korzystając z funkcji okna, z wykorzystaniem podzapytań lub instrukcji join:

- row_number:

```
-- row_number() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid, count(p2.productid) + 1 as rowno_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and
           (p2.unitprice > p1.unitprice or (p2.unitprice = p1.unitprice and p2.productid < p1.productid))
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, rowno_custom;
```

```
-- row_number() z subquery
select productid, productname, unitprice, categoryid,
    (
        select count(*) + 1
        from products p2
        where p2.categoryid = p1.categoryid
            and (
                p2.unitprice > p1.unitprice
                or (p2.unitprice = p1.unitprice and p2.productid < p1.productid)
            )
    ) as rowno_custom
from products p1
order by p1.categoryid, rowno_custom;
```

- rank:

```
-- rank() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid, count(p2.productid) + 1 as rankprice_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and p2.unitprice > p1.unitprice
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, rankprice_custom;
```

```
-- rank() z subquery
select productid, productname, unitprice, categoryid,
    (
        select count(*) + 1
        from products p2
        where p2.unitprice > p1.unitprice and p2.categoryid = p1.categoryid
    ) as rankprice_custom
from products p1
order by p1.categoryid, rankprice_custom;
```

- dense_rank:

```
-- dense_rank() z inner-join
select p1.productid, p1.productname, p1.unitprice, p1.categoryid, count(distinct p2.unitprice) + 1 as denserankprice_custom
from products p1
    left join products p2
        on p2.categoryid = p1.categoryid and p2.unitprice > p1.unitprice
group by p1.productid, p1.productname, p1.unitprice, p1.categoryid
order by p1.categoryid, denserankprice_custom;
```

```
-- dense_rank() z subquery
select productid, productname, unitprice, categoryid,
    (
        select count(distinct p2.unitprice) + 1
        from products p2
        where p2.unitprice > p1.unitprice and p2.categoryid = p1.categoryid
    ) as denserankprice_custom
from products p1
order by p1.categoryid, denserankprice_custom;
```

W celu porównania wyników uruchamiamy zapytanie łączące wszystkie te zapytania:

```
with custom_rn as (select p1.productid,
                        count(p2.productid) + 1 as rowno_custom
                    from products p1
                        left join products p2
                            on p2.categoryid = p1.categoryid and
                               (p2.unitprice > p1.unitprice or
                                (p2.unitprice = p1.unitprice and p2.productid < p1.productid))
                    group by p1.productid),
custom_rk as (select p1.productid,
                    count(p2.productid) + 1 as rankprice_custom
                from products p1
                    left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice
                    group by p1.productid),
custom_dk as (select p1.productid,
                    count(distinct p2.unitprice) + 1 as denserankprice_custom
                from products p1
                    left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice
                    group by p1.productid)
select p.productid,
       p.productname,
       p.unitprice,
       p.categoryid,
       custom_rn.rowno_custom,
       row_number() over (partition by categoryid order by p.unitprice desc, p.productid) as rowno,
       custom_rk.rankprice_custom,
       rank() over (partition by categoryid order by p.unitprice desc) as rankprice,
       custom_dk.denserankprice_custom,
       dense_rank() over (partition by categoryid order by p.unitprice desc) as denserankprice
from products p
     join custom_rn on custom_rn.productid = p.productid
     join custom_rk on custom_rk.productid = p.productid
     join custom_dk on custom_dk.productid = p.productid
order by p.categoryid, custom_rn.rowno_custom;
```

	productid	productname	unitprice	categoryid	rowno_custom	rowno	rankprice_custom	rankprice	denserankprice_custom	denserankprice
1	38	Côte de Blaye	263.5	1	1	1	1	1	1	1
2	43	Ipoh Coffee	46	1	2	2	2	2	2	2
3	2	Chang	19	1	3	3	3	3	3	3
4	1	Chai	18	1	4	4	4	4	4	4
5	35	Steeleye Stout	18	1	5	5	4	4	4	4
6	39	Chartreuse verte	18	1	6	6	4	4	4	4
7	76	LakkaLiköbri	18	1	7	7	4	4	4	4
8	70	Outback Lager	15	1	8	8	8	8	5	5
9	34	Sasquatch Ale	14	1	9	9	9	9	6	6
10	67	Laughing Lumberjack Lager	14	1	10	10	9	9	6	6
11	75	Rhönbräu Klosterbier	7.75	1	11	11	11	11	7	7
12	24	Guaraná Fantástica	4.5	1	12	12	12	12	8	8
13	63	Vegie-spread	43.9	2	1	1	1	1	1	1
14	8	Northwoods Cranberry Sauce	40	2	2	2	2	2	2	2

Wizualizacja 2: Zadanie 1 - porównanie funkcji okna z odpowiednikami (PostgreSQL)

W celu upewnienia się, że wszystkie wartości naszych odpowiedników są identyczne jak te z zapytań korzystających z funkcji okna, korzystamy z dwukierunkowego zapytania except (dzięki temu możemy sprawdzić czy w wyniku dostajemy identyczne wiersze, czy istnieją jakieś różnice):

```
with q1 as (with custom_rn as (select p1.productid,
                                count(p2.productid) + 1 as rowno
                                from products p1
                                left join products p2
                                on p2.categoryid = p1.categoryid and
                                   (p2.unitprice > p1.unitprice or
                                    (p2.unitprice = p1.unitprice and p2.productid < p1.productid))

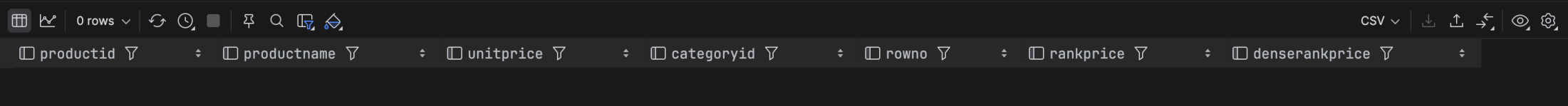
                                group by p1.productid),
    custom_rk as (select p1.productid,
                        count(p2.productid) + 1 as rankprice
                        from products p1
                        left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice

                        group by p1.productid),
    custom_dk as (select p1.productid,
                        count(distinct p2.unitprice) + 1 as denserankprice
                        from products p1
                        left join products p2
                        on p2.categoryid = p1.categoryid and
                           p2.unitprice > p1.unitprice

                        group by p1.productid)

select p.productid,
       p.productname,
       p.unitprice,
       p.categoryid,
       custom_rn.rowno,
       custom_rk.rankprice,
       custom_dk.denserankprice
from products p
      join custom_rn on custom_rn.productid = p.productid
      join custom_rk on custom_rk.productid = p.productid
      join custom_dk on custom_dk.productid = p.productid
order by p.categoryid, custom_rn.rowno),
q2 as (select p.productid,
             p.productname,
             p.unitprice,
             p.categoryid,
             row_number() over (partition by categoryid order by p.unitprice desc, p.productid) as rowno,
             rank() over (partition by categoryid order by p.unitprice desc) as rankprice,
             dense_rank() over (partition by categoryid order by p.unitprice desc) as denserankprice
             from products p
             order by p.categoryid)
(select * from q1 except select * from q2)
union all
(select * from q2 except select * from q1);
```

Wynik:



Wizualizacja 3: Zadanie 1 - weryfikacja except, pusty wynik (PostgreSQL)

Jak widać na załączonym zrzucie ekranu, wszystkie funkcje oraz nasze customowe odpowiedniki dają identyczne rezultaty (zapytanie zwraca 0 wierszy, co oznacza, wynikowy zbiór jest identyczny dla obu zapytań). W dalszych częściach konspektu porównania klauzulę except będziemy wykonywać analogicznie.

Ze względu na różnicę w wydajności podzapytania względem inner-joina, w dalszej części konspektu będziemy korzystać z metody wykorzystującej inner-joina (porównanie wykonane dla silnika Postgres oraz funkcji row_number, wydajność pozostałych funkcji ma podobną charakterystykę):

- funkcja okna:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
⌵ ⇐ Select					
⌵ Transformation (WindowAgg)		77	5.92	4.21	Parallel Aware = false; Async Capable = ...
⌵ Sort		77	4.38	4.18	Parent Relationship = Outer; Parallel Aw...
🗒 Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel Aw...

Wizualizacja 4: Zadanie 1 - plan zapytania z funkcją okna (PostgreSQL)

- inner-join:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
⌵ ⇐ Select					
⌵ Sort		77	25.08	24.88	Parallel Aware = false; Async Capable ...
⌵ Aggregate		77	22.47	21.51	Strategy = Hashed; Partial Mode = Sim...
⌵ ⚙ Hash Join		274	20.14	2.73	Parent Relationship = Outer; Parallel A...
🗒 Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel A...
⌵ Transformation (Hash)		77	1.77	1.77	Parent Relationship = Inner; Parallel Aw...
🗒 Full Scan (Seq Scan)	table: products;	77	1.77	0.0	Parent Relationship = Outer; Parallel A...

Wizualizacja 5: Zadanie 1 - plan zapytania z inner-join (PostgreSQL)

- podzapytanie:

Operation	Params	Rows	Total Cost	Startup Cost	Raw Desc
⌵ ⇐ Select					
⌵ Sort		77	201.5	201.3	Parallel Aware = false; Async Capable =...
⌵ 🗒 Full Scan (Seq Scan)	table: products;	77	198.89	0.0	Parent Relationship = Outer; Parallel Aw...
⌵ Aggregate		1	2.56	2.55	Strategy = Plain; Partial Mode = Simple;...
🗒 Full Scan (Seq Scan)	table: products;	3	2.54	0.0	Parent Relationship = Outer; Parallel Aw...

Wizualizacja 6: Zadanie 1 - plan zapytania z podzapytaniem (PostgreSQL)

Zadanie 2

Baza: Northwind, tabela product_history

Dla każdego produktu, podaj 4 najwyższe ceny tego produktu w danym roku. Zbiór wynikowy powinien zawierać:

- rok
- id produktu
- nazwę produktu
- cenę
- datę (datę uzyskania przez produkt takiej ceny)
- pozycję w rankingu
- Uporządkuj wynik wg roku, nr produktu, pozycji w rankingu

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy.

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

W przypadku zapytania wykorzystującego funkcję okna, korzystamy z funkcji dense_rank, a następnie wybieramy unikalne unikalne wartości (rok, productid, cena) z denserankprice < 4 - w rezultacie uzyskamy 4 najwyższe ceny, bez względu na występujące remisy. Ze względu na długi czas wykonania explain analyse (w przypadku zapytania z inner-join, zapytanie z funkcją okna działa bardzo szybko), zbiór wynikowy został ograniczony jedynie do produktów o productid < 10.

```
-- zapytanie wykorzystujące dense_rank()
```

```
with t as (select distinct on (extract(year from date), productid, unitprice)
           extract(year from date) as year,
           productid,
           productname,
```

```

        unitprice,
        date,
        dense_rank() over (
            partition by extract(year from date), productid
            order by unitprice desc
        )
        as denserankprice
    from product_history p)
select *
from t
where t.denserankprice <= 4 and t.productid < 10
order by year, productid, unitprice desc;

-- zapytanie bez funkcji okna (inner-join)

with t as (select distinct on (extract(year from date), productid, unitprice)
    extract(year from date) as year,
    productid,
    productname,
    unitprice,
    date
    from product_history),
dr as (select p1.year,
    p1.productid,
    p1.productname,
    p1.unitprice,
    p1.date,
    count(distinct p2.unitprice) + 1 as denserankprice
    from t p1
        left join t p2 on p2.year = p1.year and p2.productid = p1.productid and p2.unitprice > p1.unitprice
    group by p1.year, p1.productid, p1.productname, p1.unitprice, p1.date)
select *
from dr
where denserankprice <= 4 and productid < 10
order by year, productid, unitprice desc;

```


Rezultaty:

- z funkcją okna:

	year	productid	productname	unitprice	date	denserankprice
1	1940	1	Chai	27.97	1940-01-05	1
2	1940	1	Chai	27.95	1940-11-26	2
3	1940	1	Chai	27.90	1940-11-29	3
4	1940	1	Chai	27.86	1940-10-09	4
5	1940	2	Chang	28.93	1940-03-09	1
6	1940	2	Chang	28.88	1940-11-07	2
7	1940	2	Chang	28.87	1940-05-26	3
8	1940	2	Chang	28.55	1940-11-21	4
9	1940	3	Aniseed Syrup	19.97	1940-06-20	1
10	1940	3	Aniseed Syrup	19.96	1940-12-06	2
11	1940	3	Aniseed Syrup	19.88	1940-01-14	3
12	1940	3	Aniseed Syrup	19.77	1940-05-26	4
13	1940	4	Chef Anton's Cajun Seasoning	31.97	1940-03-01	1

Wizualizacja 7: Zadanie 2 - wynik dense_rank z funkcją okna (PostgreSQL)

- z inner-join:

1	1940	1	Chai	27.97	1940-01-05	1
2	1940	1	Chai	27.95	1940-11-26	2
3	1940	1	Chai	27.90	1940-11-15	3
4	1940	1	Chai	27.86	1940-10-09	4
5	1940	2	Chang	28.93	1940-03-09	1
6	1940	2	Chang	28.88	1940-11-07	2
7	1940	2	Chang	28.87	1940-05-26	3
8	1940	2	Chang	28.55	1940-11-21	4
9	1940	3	Aniseed Syrup	19.97	1940-06-20	1
10	1940	3	Aniseed Syrup	19.96	1940-12-06	2
11	1940	3	Aniseed Syrup	19.88	1940-07-28	3
12	1940	3	Aniseed Syrup	19.77	1940-05-26	4
13	1940	4	Chef Anton's Cajun Seasoning	31.97	1940-03-01	1

Wizualizacja 8: Zadanie 2 - wynik dense_rank z inner-join (PostgreSQL)

W celu zweryfikowania poprawności zapytania niekorzystającego z funkcji okna, tworzymy zapytanie korzystające z dwukierunkowego except. Ze względu na fakt, że ceny o danej randze mogły zostać zaobserwowane w różnych dniach, a zadanie nie specyfikowała, która data powinna być zawarta w zbiorze wynikowym, data nie jest brana pod uwagę przy porównywaniu dwóch zbiorów wynikowych.

Wyniki:

Wizualizacja 9: Zadanie 2 - weryfikacja except, pusty wynik (PostgreSQL)

W przypadku uwzględnienia daty w zapytaniu porównującym te dwa podejścia, obserwujemy pewne różnice (natomiast dla każdego produktu, ceny o danej randze są identyczne, wiersze różnią się wyłącznie datą wystąpienia):

	year	productid	productname	unitprice	date	denserankprice
1	1940	1	Chai	27.90	1940-11-29	3
2	1940	3	Aniseed Syrup	19.88	1940-01-14	3
3	1940	1	Chai	27.90	1940-11-15	3
4	1940	3	Aniseed Syrup	19.88	1940-07-28	3
5	1941	7	Uncle Bob's Organic Dried Pears	39.67	1941-10-03	3
6	1941	8	Northwoods Cranberry Sauce	49.83	1941-03-20	2
7	1941	8	Northwoods Cranberry Sauce	49.83	1941-02-10	2
8	1941	7	Uncle Bob's Organic Dried Pears	39.67	1941-04-26	3
9	1942	7	Uncle Bob's Organic Dried Pears	39.99	1942-10-14	1
10	1942	3	Aniseed Syrup	19.98	1942-01-30	1
11	1942	7	Uncle Bob's Organic Dried Pears	39.90	1942-08-09	4
12	1942	5	Chef Anton's Gumbo Mix	31.31	1942-02-19	1
13	1942	7	Uncle Bob's Organic Dried Pears	39.90	1942-11-23	4
14	1942	5	Chef Anton's Gumbo Mix	31.31	1942-05-12	1
15	1942	3	Aniseed Syrup	19.98	1942-11-22	1
16	1942	7	Uncle Bob's Organic Dried Pears	39.99	1942-07-13	1
17	1943	6	Grandma's Boysenberry Spread	34.87	1943-11-03	3

Wizualizacja 10: Zadanie 2 - except z datą, widoczne różnice (PostgreSQL)

Zapytania dla MSSQL oraz SQLite są generalnie identyczne, z dokładnością do wyciągania roku z daty:

```
-- postgres
extract(year from date) as year
```

```
-- mssql
year(date) as year
```

```
-- sqlite
strftime('%Y', date) as year
```

Porównanie planów zapytań (Postgres):

- zapytanie z dense_rank

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning = {"Shar...
⌵ Unknown (Incremental Sort)		55417	2988.0	138711.88	474.498	86143.07	236.484	Parallel Aware = f...
⌵ 🗲 Access (Subquery Scan)		55417	2988.0	137204.15	473.435	86139.91	233.995	Parent Relationshi...
⌵ Unique		166251	249180.0	135126.02	460.623	86139.91	233.924	Parent Relationshi...
⌵ Unknown (Incremental Sort)		275891	270000.0	133056.83	437.155	86139.91	233.924	Parent Relationshi...
⌵ Transformation (WindowA		275891	270000.0	124479.14	352.849	86139.67	233.636	Parent Relationshi...
⌵ Unknown (Gather Merç		275891	270000.0	118271.59	284.779	86139.54	233.584	Parent Relationshi...
⌵ Sort		114955	90000.0	85426.9	198.862	85139.51	192.912	Parent Relationshi...
🗲 Full Scan (Se table: product_history;		114955	90000.0	71154.64	115.058	0.0	7.88	Parent Relationshi...

Wizualizacja 11: Zadanie 2 - plan zapytania z dense_rank (PostgreSQL)

- zapytanie z inner-join

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning = {"Share...
⌵ Unknown (Incremental Sort)		7566	2988.0	552492.36	25296.023	526300.58	1252.499	Parallel Aware = fa...
⌵ Unique		231000	2077307.0	526265.15	1553.258	239902.58	956.573	Parent Relationshi...
⌵ Unknown (Gather Merge)		2310000	2310000.0	508940.15	1333.916	239902.58	956.572	Parent Relationshi...
⌵ Sort		962500	770000.0	241308.81	971.498	238902.56	923.968	Parent Relationshi...
🗲 Full Scan (Seq Sc table: product_history;		962500	770000.0	70867.25	191.0	0.0	85.046	Parent Relationshi...
⌵ Aggregate		7566	2988.0	25991.86	25288.906	0.93	987.16	Strategy = Sorted;...
⌵ Unknown (Incremental Sor		148225	4.188187E+7	23409.08	20710.593	0.93	956.876	Parent Relationshi...
⌵ ⚡ Merge Join		148225	4.188187E+7	19096.0	10781.271	0.0	956.615	Parent Relationshi...
🗲 Access (CTE Sca table: t;		77000	249180.0	5197.5	1701.285	0.0	956.599	Parent Relationshi...
⌵ 🗲 Temporary (Mate		231000	85836212	5197.5	2004.698	0.0	0.002	Parent Relationshi...
🗲 Access (CTE † table: t;		231000	2073966.0	4620.0	188.992	0.0	0.0	Parent Relationshi...

Wizualizacja 12: Zadanie 2 - plan zapytania z inner-join (PostgreSQL)

Wnioski:

- zapytanie wykorzystujące funkcję okna (dense_rank), charakteryzuje się ponad 4-krotnie niższym kosztem (total cost)
- faktyczny czas wykonania zapytania jest ~50 razy krótszy (actual total time)
- wiersze w zapytaniu z inner-join musiały się zmaterializować, w rezultacie tworzonych jest 85 milionów wierszy pośrednich
- oczekiwana liczba wierszy (rows) jest mocno niedoszacowana względem faktycznej liczby wierszy (actual rows), co może skutkować złym wyborem planu wykonania zapytania.

Alternatywne zapytanie korzystające z subquery nie wykonało się w rozsądnym czasie, nawet na ograniczonym zbiorze danych.

Porównanie planów zapytań - MSSQL:

- zapytanie z dense_rank

Operation	...	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
⌵ ⌵ Select ⚠		80598.5		21.091		BatchModeOnRowStoreUs...
⌵ Transformation (Gather Streams - Parallelism)		80598.5	3264	21.091	74.0	AvgRowSize = 55; Estimate...
⌵ Sort		80598.5	3264	20.5118	4.0	AvgRowSize = 55; Estimate...
⌵ Aggregate (Aggregate - Hash Match)		80598.5	3264	20.3601	2.0	AvgRowSize = 55; Estimate...
⌵ Filter		80598.5	3264	20.2202	0.0	AvgRowSize = 55; Estimate...
⌵ ▢ Value (Compute Scalar)		268662	270000	20.1879	2.0	AvgRowSize = 55; Estimate...
⌵ Unknown (Window Aggregate)		268662	270000	20.1873	0.0	AvgRowSize = 51; Estimate...
⌵ Sort		268662	270000	20.1866	23.0	AvgRowSize = 47; Estimate...
⌵ ▢ Value (Compute Scalar)		268662	270000	19.6296	3.0	AvgRowSize = 47; Estimate...
🔑 Full Index Scan (Clustered Index Scan)	ta...	268662	270000	19.3517	36.0	AvgRowSize = 43; Estimate...

Wizualizacja 13: Zadanie 2 - plan zapytania z dense_rank (MSSQL)

- zapytanie z inner-join

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
⌵ ⌵ Select ⚠		11028.4		51.9911		BatchModeOnRowStoreUsed ...
⌵ Transformation (Gather Streams - Parallelism)		11028.4	3264	51.9911	6467.0	AvgRowSize = 51; EstimateCP...
⌵ Sort		11028.4	3264	51.888	8.0	AvgRowSize = 51; EstimateCP...
⌵ ▢ Value (Compute Scalar)		11028.4	3264	51.87	0.0	AvgRowSize = 51; EstimateCP...
⌵ Filter		11028.4	3264	51.87	0.0	AvgRowSize = 51; EstimateCP...
⌵ ▢ Value (Compute Scalar)		268661	270000	51.8654	0.0	AvgRowSize = 51; EstimateCP...
⌵ Aggregate (Aggregate - Hash Match)		268661	270000	51.8654	5929.0	AvgRowSize = 51; EstimateCP...
⌵ ⌵ Hash Join (Left Outer Join - Hash		2.30998E+6	49079870	51.052	311.0	AvgRowSize = 56; EstimateCP...
⌵ Aggregate (Aggregate - Hash Ma		268661	270000	20.0224	97.0	AvgRowSize = 47; EstimateCP...
⌵ ▢ Value (Compute Scalar)		268662	270000	19.6296	3.0	AvgRowSize = 47; EstimateCP...
🔑 Full Index Scan (Cluster table: [dbo].[...		268662	270000	19.3517	24.0	AvgRowSize = 43; EstimateCP...
⌵ Aggregate (Aggregate - Hash Ma		2.30998E+6	270000	22.7205	104.0	AvgRowSize = 24; EstimateCP...
⌵ Filter		2.31E+6	270000	19.3569	1.0	AvgRowSize = 64; EstimateCP...
⌵ ▢ Value (Compute Scalar)		2.31E+6	2310000	19.3569	20.0	AvgRowSize = 64; EstimateCP...
🔑 Full Index Scan (Clus table: [dbo].[...		2.31E+6	2310000	19.3512	31.0	AvgRowSize = 47; EstimateCP...

Wizualizacja 14: Zadanie 2 - plan zapytania z inner-join (MSSQL)

Wnioski:

- dla ograniczonego zapytania, całkowity koszt zapytania z funkcją okna jest około 2,5 raza niższy względem zapytania z joinem, a faktyczny czas zapytania jest ~90 razy niższy.
- zapytanie jest zdecydowanie szybsze w porównaniu do Postgresa (~7 razy dla funkcji okna oraz ~4 razy dla zapytania z joinem)
- zapytanie na nieograniczonym zbiorze wykonało się w rozsądnym czasie zarówno dla zapytania z funkcją okna oraz joinem (~200 razy niższy całkowity koszt zapytania, zapytanie z funkcją okna wykonywało się około 400ms, zapytaniem z joinem około 1,5 minuty)

Porównanie wydajności i planów zapytań - SQLite:

- z funkcją dense_rank:

Operation	Params	Raw Desc
⌵ ⌵ Select		
⌵ Operation (CO-ROUTINE)		CO-ROUTINE t
⌵ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
⌵ Full Scan	table: product_history;	SCAN product_history
Order By		USE TEMP B-TREE FOR ORDER BY
⌵ Full Scan	table: (subquery-3);	SCAN (subquery-3)
Unique		USE TEMP B-TREE FOR DISTINCT
⌵ Full Scan	table: t;	SCAN t
Order By		USE TEMP B-TREE FOR ORDER BY

Wizualizacja 15: Zadanie 2 - plan zapytania z dense_rank (SQLite)

- z inner-join:

Operation	Params	Raw Desc
⌵ ⌵ Select		
⌵ Operation (CO-ROUTINE)		CO-ROUTINE dr
⌵ Operation (MATERIALIZE)		MATERIALIZE t
⌵ Full Scan	table: product_history;	SCAN product_history
Unique		USE TEMP B-TREE FOR DISTINCT
⌵ Full Scan	table: p1;	SCAN p1
Unknown		BLOOM FILTER ON p2 (year=? AND productid=?)
🔑 Index Scan	table: p2; index: (year=?;	SEARCH p2 USING AUTOMATIC COVERING INDEX (year=? AND productid=?)
⌵ Group By		USE TEMP B-TREE FOR GROUP BY
Transformation (count(DISTINCT))		USE TEMP B-TREE FOR count(DISTINCT)
⌵ Full Scan	table: dr;	SCAN dr
Order By		USE TEMP B-TREE FOR ORDER BY

Wizualizacja 16: Zadanie 2 - plan zapytania z inner-join (SQLite)

Wnioski:

- zapytanie korzystające z funkcji okna dla ograniczonego zbioru danych wykonuje się poniżej 1 sekundy, w przypadku zapytania z joinem czas wykonania zapytania wynosi około 1 minuty (nie są to jednak dokładne wyniki ze względu na paginację, a SQLite nie posiada opcji explain_analyze z dokładną informacją odnośnie faktycznego czasu wykonania zapytania)
- w planie zapytania z inner-joinem widzimy, że dane muszą się faktycznie zmaterializować

Zadanie 3

Funkcje lag(), lead()

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje lag(), lead()

```
select productid,
       productname,
       categoryid,
       date,
       unitprice,
       lag(unitprice) over (partition by productid order by date)
       as previousprodprice,
       lead(unitprice) over (partition by productid order by date)
       as nextprodprice
from product_history
where productid = 1
      and year(date) = 2022
order by date;
```

```
with t as (select productid,
                  productname,
                  categoryid,
                  date,
                  unitprice,
                  lag(unitprice) over (partition by productid
                                      order by date) as previousprodprice,
                  lead(unitprice) over (partition by productid
                                       order by date) as nextprodprice
            from product_history)
select *
from t
where productid = 1
      and year(date) = 2022
order by date;
```

Jak działają funkcje lag(), lead()?

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna

Do analizy użyj wybranego systemu/bazy danych (wybierz MS SQLserver, Postgres lub SQLite).

Wyniki:

Początek wyniku:

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
1	1	Chai	1	2022-01-01	13.63	<null>	13.01
2	1	Chai	1	2022-01-02	13.01	13.63	10.55
3	1	Chai	1	2022-01-03	10.55	13.01	15.47
4	1	Chai	1	2022-01-04	15.47	10.55	12.64
5	1	Chai	1	2022-01-05	12.64	15.47	11.39
6	1	Chai	1	2022-01-06	11.39	12.64	14.89
7	1	Chai	1	2022-01-07	14.89	11.39	18.34
8	1	Chai	1	2022-01-08	18.34	14.89	15.81
9	1	Chai	1	2022-01-09	15.81	18.34	17.33
10	1	Chai	1	2022-01-10	17.33	15.81	12.93
11	1	Chai	1	2022-01-11	12.93	17.33	24.24
12	1	Chai	1	2022-01-12	24.24	12.93	14.82
13	1	Chai	1	2022-01-13	14.82	24.24	10.02
14	1	Chai	1	2022-01-14	10.02	14.82	10.81
15	1	Chai	1	2022-01-15	10.81	10.02	19.47

Wizualizacja 17: Zadanie 3 - Początek wyniku

Koniec wyniku:

	productid	productname	categoryid	date	unitprice	previousprodprice	nextprodprice
41	1	Chai	1	2022-02-10	12.11	19.07	11.25
42	1	Chai	1	2022-02-11	11.25	12.11	12.29
43	1	Chai	1	2022-02-12	12.29	11.25	17.95
44	1	Chai	1	2022-02-13	17.95	12.29	26.47
45	1	Chai	1	2022-02-14	26.47	17.95	10.41
46	1	Chai	1	2022-02-15	10.41	26.47	17.58
47	1	Chai	1	2022-02-16	17.58	10.41	27.75
48	1	Chai	1	2022-02-17	27.75	17.58	10.33
49	1	Chai	1	2022-02-18	10.33	27.75	22.21
50	1	Chai	1	2022-02-19	22.21	10.33	<null>

Wizualizacja 18: Zadanie 3 - Koniec wyniku

Według sygnatury z dokumentacji PostgreSQL funkcja lag() zwraca wartość z poprzedniego wiersza co do offsetu, a lead() zwraca wartość z następnego wiersza co do offsetu. W przypadku braku takiego wiersza zwracana jest wartość domyślna. W naszym przypadku (brak podania offsetu i wartości domyślnej) offset jest równy 1, a wartość domyślna jest równa NULL. Oznacza to, że funkcja lag() zwraca cenę produktu z poprzedniego dnia, a lead() zwraca cenę produktu z następnego dnia. W przypadku pierwszego wiersza (brak poprzedniego dnia) funkcja lag() zwraca NULL, a w przypadku ostatniego wiersza (brak następnego dnia) funkcja lead() zwraca NULL.

Podejście bez funkcji okna:

```
-- podzapytanie
select ph.productid,
       ph.productname,
       ph.categoryid,
       ph.date,
       ph.unitprice,
       (select ph2.unitprice
        from product_history ph2
        where ph2.productid = ph.productid
          and ph2.date < ph.date
          and extract(year from date) = 2022
        order by ph2.date desc
        limit 1) as previousprodprice,
       (select ph3.unitprice
        from product_history ph3
        where ph3.productid = ph.productid
          and ph3.date > ph.date
          and extract(year from date) = 2022
        order by ph3.date
        limit 1) as nextprodprice
from product_history ph
where ph.productid = 1
      and extract(year from ph.date) = 2022
order by ph.date;

-- joiny
select ph.productid,
       ph.productname,
       ph.categoryid,
       ph.date,
       ph.unitprice,
       ph_prev.unitprice as previousprodprice,
       ph_next.unitprice as nextprodprice
from product_history ph
      left join product_history ph_prev on ph.productid = ph_prev.productid
      and ph_prev.date = (select max(date)
                        from product_history
                        where productid = ph.productid
                          and date < ph.date
                          and extract(year from date) = 2022)
      left join product_history ph_next on ph.productid = ph_next.productid
      and ph_next.date = (select min(date)
                        from product_history
                        where productid = ph.productid
                          and date > ph.date
                          and extract(year from date) = 2022)

where ph.productid = 1
      and extract(year from ph.date) = 2022
order by ph.date;
```

Porównanie wyników klazulą except w obie strony dało pusty zbiór wynikowy, co oznacza, że wyniki są takie same dla wszystkich trzech podejść.

Porównanie planów wykonania dla różnych podejść:

- funkcje okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ↵ Select								Planning Time = 0.07...
▼ Transformation (WindowAgg)		144	50	76701.2	63.576	76681.55	61.432	Parallel Aware = false...
▼ Unknown (Gather Merge)		144	50	76698.32	63.552	76681.55	61.424	Parent Relationship ...
▼ Sort		60	17	75681.67	53.421	75681.52	53.421	Parent Relationship ...
Full Scan (Seq Sc	table: product_history;	60	17	75679.75	53.381	0.0	53.325	Parent Relationship ...

Wizualizacja 19: Zadanie 3 - Plan wykonania (funkcje okna)

- podzapytanie:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ↵ Select								Planning Time = 0.087;...
▼ Unknown (Gather Merge)		144	50	3.032713904E7	13220.903	76681.55	396.597	Parallel Aware = false;A...
▼ Sort		60	17	75681.67	121.849	75681.52	121.845	Parent Relationship = ...
Full Scan (Seq Scan)	table: product_history;	60	17	75679.75	121.805	0.0	121.76	Parent Relationship = ...
▼ Transformation (Limit)		1	1	105036.25	135.728	105036.25	135.728	Parent Relationship = S...
▼ Sort		50	1	105036.38	135.726	105036.25	135.726	Parent Relationship = ...
Full Scan (Seq Scan)	table: product_history;	50	24	105036.0	135.718	0.0	135.592	Parent Relationship = ...
▼ Transformation (Limit)		1	1	105036.25	126.047	105036.25	126.046	Parent Relationship = S...
▼ Sort		50	1	105036.38	126.044	105036.25	126.044	Parent Relationship = ...
Full Scan (Seq Scan)	table: product_history;	50	24	105036.0	126.037	0.0	125.968	Parent Relationship = ...

Wizualizacja 20: Zadanie 3 - Plan wykonania (podzapytanie)

- joiny:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ↵ Select								Planning Time = 0.1...
▼ Sort		314	50	6.07121431472E9	13294.477	6.07121431393E9	13293.566	Parallel Aware = fal...
▼ Merge Join		314	50	6.07121430091E9	13294.467	3.03209862301E9	13293.547	Parent Relationship...
▼ Sort		213	50	3.03202175743E9	13212.004	3.03202175689E9	13211.942	Parent Relationship...
▼ Merge Join		213	50	3.03202174866E9	13211.947	153565.43	7128.259	Parent Relationship...
▼ Sort		144	50	76699.67	6916.324	76699.31	6916.259	Parent Relationship...
▼ Unknown (Gather)		144	50	76694.15	6916.244	1000.0	365.505	Parent Relationship...
Full Scan (Seq Scan)	table: product_history;	60	17	75679.75	164.907	0.0	164.859	Parent Relationship...
▼ Aggregate		1	1	105036.13	133.775	105036.12	133.775	Strategy = Plain;Par...
Full Scan (Seq Scan)	table: product_history;	50	24	105036.0	133.77	0.0	133.645	Parent Relationship...
▼ Sort		28721	30000	76937.92	84.846	76866.11	84.104	Parent Relationship...
▼ Unknown (Gather)		28721	30000	74739.35	80.877	1000.0	35.124	Parent Relationship...
Full Scan (Seq Scan)	table: product_history;	11967	10000	70867.25	70.115	0.0	44.656	Parent Relationship...
▼ Aggregate		1	1	105036.13	124.195	105036.12	124.195	Strategy = Plain;Par...
Full Scan (Seq Scan)	table: product_history;	50	24	105036.0	124.191	0.0	124.125	Parent Relationship...
▼ Sort		28721	30000	76937.92	81.6	76866.11	80.05	Parent Relationship...
▼ Unknown (Gather)		28721	30000	74739.35	77.68	1000.0	34.378	Parent Relationship...
Full Scan (Seq Scan)	table: product_history;	11967	10000	70867.25	68.514	0.0	42.975	Parent Relationship...

Wizualizacja 21: Zadanie 3 - Plan wykonania (joiny)

Wnioski:

- podejście z funkcjami okna jest znacznie szybsze niż podzapytanie i joiny, co widać po czasie wykonania i kosztach
- funkcja okna wykonuje jeden skan tabeli product_history, natomiast podzapytanie i joiny wykonują wiele skanów tej tabeli (po jednym dla każdego wiersza), co jest przyczyną dłuższego czasu wykonania

Zadanie 4

Baza: Northwind, tabele customers, orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- nazwę klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- nr poprzedniego zamówienia danego klienta,
- datę poprzedniego zamówienia danego klienta,
- wartość poprzedniego zamówienia danego klienta.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite.

Wyniki:

```
-- PostgreSQL
with ordersummary as (select c.companyname,
                             o.orderid,
                             o.orderdate,
                             sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                        from orders o
                             join customers c on o.customerid = c.customerid
                             join orderdetails od on o.orderid = od.orderid
                        group by c.companyname, o.orderid, o.orderdate, o.freight)

select companyname,
      orderid,
       orderdate,
       ordertotal,
       lag(orderid) over (partition by companyname order by orderdate, orderid)   as prevorderid,
       lag(orderdate) over (partition by companyname order by orderdate, orderid) as prevorderdate,
       lag(ordertotal) over (partition by companyname order by orderdate, orderid) as prevordertotal
from ordersummary
order by companyname, orderdate;
```

	companyname	orderid	orderdate	ordertotal	prevorderid	prevorderdate	prevordertotal
1	Alfreds Futterkiste	10643	1997-08-25	843.959981918335	<null>	<null>	<null>
2	Alfreds Futterkiste	10692	1997-10-03	939.0200309753418	10643	1997-08-25	843.959981918335
3	Alfreds Futterkiste	10702	1997-10-13	353.9400005340576	10692	1997-10-03	939.0200309753418
4	Alfreds Futterkiste	10835	1998-01-15	915.3299987018108	10702	1997-10-13	353.9400005340576
5	Alfreds Futterkiste	10952	1998-03-16	511.6199948191643	10835	1998-01-15	915.3299987018108
6	Alfreds Futterkiste	11011	1998-04-09	934.7099996432662	10952	1998-03-16	511.6199948191643
7	Ana Trujillo Emparedados y hel...	10308	1996-09-18	90.40999925136566	<null>	<null>	<null>
8	Ana Trujillo Emparedados y hel...	10625	1997-08-08	523.6500015258789	10308	1996-09-18	90.40999925136566
9	Ana Trujillo Emparedados y hel...	10759	1997-11-28	331.98999977111816	10625	1997-08-08	523.6500015258789
10	Ana Trujillo Emparedados y hel...	10926	1998-03-04	554.3199892044067	10759	1997-11-28	331.98999977111816
11	Antonio Moreno Taquería	10365	1996-11-27	425.1999816894531	<null>	<null>	<null>
12	Antonio Moreno Taquería	10507	1997-04-15	796.5124955102801	10365	1996-11-27	425.1999816894531
13	Antonio Moreno Taquería	10535	1997-05-13	1956.48999369666	10507	1997-04-15	796.5124955102801
14	Antonio Moreno Taquería	10573	1997-06-19	2166.8399772644043	10535	1997-05-13	1956.48999369666
15	Antonio Moreno Taquería	10677	1997-09-22	817.3949828332663	10573	1997-06-19	2166.8399772644043
16	Antonio Moreno Taquería	10682	1997-09-25	411.63000106811523	10677	1997-09-22	817.3949828332663
17	Antonio Moreno Taquería	10856	1998-01-28	718.4300003051758	10682	1997-09-25	411.63000106811523
18	Antonio Moreno Taquería	10355	1996-11-15	521.9500079154968	<null>	<null>	<null>
		10383	1996-12-16	933.2399978637695	10355	1996-11-15	521.9500079154968

Wizualizacja 22: Zadanie 4 - Wynik

Dla każdego klienta widzimy jego zamówienia wraz z informacjami o poprzednim zamówieniu. W przypadku pierwszego zamówienia danego klienta, kolumny dotyczące poprzedniego zamówienia będą zawierały wartość NULL.

Podejścia bez funkcji okna:

```
-- podzapytanie
with ordersummary as (select c.companyname,
                             o.customerid,
                             o.orderid,
                             o.orderdate,
                             sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                        from orders o
                             join customers c on o.customerid = c.customerid
                             join orderdetails od on o.orderid = od.orderid
                        group by c.companyname, o.customerid, o.orderid, o.orderdate, o.freight)

select t1.companyname,
       t1.orderid,
       t1.orderdate,
       t1.ordertotal,
       (select t2.orderid
        from ordersummary t2
        where t2.customerid = t1.customerid
              and (t2.orderdate < t1.orderdate or (t2.orderdate = t1.orderdate and t2.orderid < t1.orderid))
        order by t2.orderdate desc, t2.orderid desc
        limit 1) as prevorderid,
       (select t2.orderdate
        from ordersummary t2
        where t2.customerid = t1.customerid
              and (t2.orderdate < t1.orderdate or (t2.orderdate = t1.orderdate and t2.orderid < t1.orderid))
        order by t2.orderdate desc, t2.orderid desc
        limit 1) as prevorderdate,
       (select t2.ordertotal
        from ordersummary t2
        where t2.customerid = t1.customerid
              and (t2.orderdate < t1.orderdate or (t2.orderdate = t1.orderdate and t2.orderid < t1.orderid))
        order by t2.orderdate desc, t2.orderid desc
        limit 1) as prevordertotal
from ordersummary t1
order by t1.companyname, t1.orderdate, t1.orderid;

-- joiny
with ordersummary as (select c.companyname,
                             o.customerid,
                             o.orderid,
                             o.orderdate,
                             sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                        from orders o
                             join customers c on o.customerid = c.customerid
                             join orderdetails od on o.orderid = od.orderid
                        group by c.companyname, o.customerid, o.orderid, o.orderdate, o.freight),
ordermapping as (select t1.orderid,
                       (select t2.orderid
                        from ordersummary t2
                        where t2.customerid = t1.customerid
                              and (t2.orderdate < t1.orderdate or
                                   (t2.orderdate = t1.orderdate and t2.orderid < t1.orderid))
                        order by t2.orderdate desc, t2.orderid desc
                        limit 1) as previd
                  from ordersummary t1)

select curr.companyname,
       curr.orderid,
       curr.orderdate,
       curr.ordertotal,
       prev.orderid as prevorderid,
       prev.orderdate as prevorderdate,
       prev.ordertotal as prevordertotal
from ordersummary curr
     join ordermapping m on curr.orderid = m.orderid
     left join ordersummary prev on m.previd = prev.orderid
order by curr.companyname, curr.orderdate, curr.orderid;
```

Porównanie wyników klazulą except w obie strony dało pusty zbiór wynikowy, co oznacza, że wyniki są takie same dla wszystkich trzech podejść.

Porównanie planów wykonania dla różnych podejść:

- funkcje okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ⚙ Select								Planning Time = 0...
▼ Transformation (WindowAgg)		2155	830	346.67	1.949	287.4	1.674	Parallel Aware = f...
▼ Sort		2155	830	292.79	1.692	287.4	1.671	Parent Relationshi...
▼ 🗒 Access (Subquery Scan)		2155	830	168.09	1.339	119.6	1.236	Parent Relationshi...
▼ Aggregate		2155	830	146.54	1.301	119.6	1.235	Strategy = Hashe...
▼ ⚡ Hash Join		2155	2155	81.89	0.868	36.72	0.169	Parent Relationshi...
▼ ⚡ Hash Join		2155	2155	71.92	0.549	32.67	0.144	Parent Relationshi...
🗒 Full Scan (Seq Sc table: orderdetails;		2155	2155	33.55	0.086	0.0	0.002	Parent Relationshi...
▼ Transformation (Hasl		830	830	22.3	0.139	22.3	0.139	Parent Relationshi...
🗒 Full Scan (Seq table: orders;		830	830	22.3	0.076	0.0	0.002	Parent Relationshi...
▼ Transformation (Hash)		91	91	2.91	0.023	2.91	0.023	Parent Relationshi...
🗒 Full Scan (Seq Sc table: customers;		91	91	2.91	0.014	0.0	0.007	Parent Relationshi...

Wizualizacja 23: Zadanie 4 - Plan wykonania (funkcje okna)

- podzapytanie:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ⚙ Select								Planning Time = 0.2...
▼ Sort		2155	830	418422.05	112.606	418416.67	112.585	Parallel Aware = fals...
▼ Aggregate		2155	830	146.54	15.375	119.6	15.257	Strategy = Hashed;...
▼ ⚡ Hash Join		2155	2155	81.89	14.702	36.72	13.768	Parent Relationship ...
▼ ⚡ Hash Join		2155	2155	71.92	0.781	32.67	0.233	Parent Relationship ...
🗒 Full Scan (Seq Scan) table: orderdetails;		2155	2155	33.55	0.129	0.0	0.003	Parent Relationship ...
▼ Transformation (Hash)		830	830	22.3	0.22	22.3	0.22	Parent Relationship ...
🗒 Full Scan (Seq Scan) table: orders;		830	830	22.3	0.105	0.0	0.005	Parent Relationship ...
▼ Transformation (Hash)		91	91	2.91	13.526	2.91	13.526	Parent Relationship ...
🗒 Full Scan (Seq Scan) table: customers;		91	91	2.91	13.505	0.0	13.495	Parent Relationship ...
▼ 🗒 Access (CTE Scan) table: ordersummary;		2155	830	418150.81	112.19	0.0	15.798	Parent Relationship ...
▼ Transformation (Limit)		1	1	64.67	0.039	64.67	0.039	Parent Relationship ...
▼ Sort		4	1	64.68	0.039	64.67	0.039	Parent Relationship ...
🗒 Access (CTE Scan) table: ordersummary;		4	6	64.65	0.038	0.0	0.012	Parent Relationship ...
▼ Transformation (Limit)		1	1	64.67	0.039	64.67	0.039	Parent Relationship ...
▼ Sort		4	1	64.68	0.038	64.67	0.038	Parent Relationship ...
🗒 Access (CTE Scan) table: ordersummary;		4	6	64.65	0.038	0.0	0.012	Parent Relationship ...
▼ Transformation (Limit)		1	1	64.67	0.039	64.67	0.039	Parent Relationship ...
▼ Sort		4	1	64.68	0.039	64.67	0.039	Parent Relationship ...
🗒 Access (CTE Scan) table: ordersummary;		4	6	64.65	0.038	0.0	0.012	Parent Relationship ...

Wizualizacja 24: Zadanie 4 - Plan wykonania (podzapytanie)

- joiny:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⚡ Select								Planning Time = 0....
⌵ Sort		250196	830	1690104.02	216.839	1689478.53	216.816	Parallel Aware = fa...
⌵ Aggregate		2155	830	146.54	190.005	119.6	189.935	Strategy = Hashe...
⌵ ⚡ Hash Join		2155	2155	81.89	189.645	36.72	189.192	Parent Relationshi...
⌵ ⚡ Hash Join		2155	2155	71.92	0.383	32.67	0.124	Parent Relationshi...
⏏ Full Scan (Seq Scan)	table: orderdetails;	2155	2155	33.55	0.072	0.0	0.003	Parent Relationshi...
⌵ Transformation (Hash)		830	830	22.3	0.11	22.3	0.109	Parent Relationshi...
⏏ Full Scan (Seq Scan)	table: orders;	830	830	22.3	0.055	0.0	0.004	Parent Relationshi...
⌵ Transformation (Hash)		91	91	2.91	189.061	2.91	189.061	Parent Relationshi...
⏏ Full Scan (Seq Scan)	table: customers;	91	91	2.91	189.045	0.0	189.037	Parent Relationshi...
⌵ ⚡ Hash Join		250196	830	1650648.04	216.461	394.87	216.246	Parent Relationshi...
⌵ ⚡ Merge Join		23220	830	1641748.6	26.171	324.83	26.036	Parent Relationshi...
⌵ Sort		2155	830	167.8	25.925	162.42	25.905	Parent Relationshi...
⌵ ⏏ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	25.807	0.0	0.047	Parent Relationshi...
⌵ Transformation (Limit)		1	1	64.67	0.031	64.67	0.031	Parent Relationshi...
⌵ Sort		4	1	64.68	0.031	64.67	0.031	Parent Relationshi...
⏏ Access (CTE Scan)	table: ordersummary;	4	6	64.65	0.03	0.0	0.009	Parent Relationshi...
⌵ Sort		2155	819	167.8	0.143	162.42	0.124	Parent Relationshi...
⏏ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	0.043	0.0	0.002	Parent Relationshi...
⌵ Transformation (Hash)		2155	830	43.1	190.203	43.1	190.202	Parent Relationshi...
⏏ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	190.122	0.0	189.937	Parent Relationshi...

Wizualizacja 25: Zadanie 4 - Plan wykonania (joiny)

Wnioski:

- w przypadku funkcji okna konieczne było, aby grupować dane po orderdate i orderid, aby dobrze obsłużyć kolejność zamówień, co spowodowało, że próba otrzymania tego samego wyniku bez funkcji okna okazała się trudna do napisania
- kod z funkcjami okna jest znacznie bardziej czytelny, łatwiejszy oraz zwięzły do napisania niż kod z podzapytaniem i joinami, co jest dodatkową zaletą funkcji okna
- czasy wykonania są bardzo małe (<1s), ale warto zwrócić uwagę na koszty, które są znacznie większe dla podejścia z podzapytaniem i joinami niż dla podejścia z funkcjami okna

Zadanie 5

Funkcje first_value(), last_value()

Baza: Northwind, tabele customers, orders, order details

Wykonaj polecenia, zaobserwuj wynik. Jak działają funkcje first_value(), last_value().

Skomentuj uzyskane wyniki. Czy funkcja first_value pokazuje w tym przypadku najdroższy produkt w danej kategorii, czy funkcja last_value() pokazuje najtańszy produkt?

Co jest przyczyną takiego działania funkcji last_value.

Co trzeba zmienić żeby funkcja last_value pokazywała najtańszy produkt w danej kategorii?

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite)

```
select productid, productname, unitprice, categoryid,
       first_value(productname) over (partition by categoryid
order by unitprice desc) first,
       last_value(productname) over (partition by categoryid
order by unitprice desc) last
from products
order by categoryid, unitprice desc;
```

Wyniki:

Analiza została przeprowadzona z wykorzystaniem bazy danych PostgreSQL.

productid	productname	unitprice	categoryid	first	last
38	Côte de Blaye	263.5	1	Côte de Blaye	Côte de Blaye
43	Ipoh Coffee	46	1	Côte de Blaye	Ipoh Coffee
2	Chang	19	1	Côte de Blaye	Chang
1	Chai	18	1	Côte de Blaye	Chartreuse verte
76	Lakkalikööri	18	1	Côte de Blaye	Chartreuse verte
35	Steeleye Stout	18	1	Côte de Blaye	Chartreuse verte
39	Chartreuse verte	18	1	Côte de Blaye	Chartreuse verte
70	Outback Lager	15	1	Côte de Blaye	Outback Lager
34	Sasquatch Ale	14	1	Côte de Blaye	Laughing Lumberjack Lager
67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Laughing Lumberjack Lager
75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Rhönbräu Klosterbier
24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
63	Vegie-spread	43.9	2	Vegie-spread	Vegie-spread

Wizualizacja 26: Zadanie 5 - domyślny wynik first_value/last_value (PostgreSQL)

Funkcja first_value() zwraca pierwszą wartość w danym oknie zgodnie z zadaną kolejnością, funkcja last_value() zwraca ostatnią wartość w danym oknie zgodnie z zadaną kolejnością, przy czym zakres funkcji last_value jest od początku do **aktualnego wiersza** (rows between unbounded preceding and current row). W naszym przypadku, funkcja first_value() faktycznie będzie pokazywała najdroższy produkt w danej kategorii, natomiast funkcja last_value() będzie zawsze pokazywała produkt z danego wiersza (bo on jest najtańszy licząc od początku do aktualnego wiersza). W celu wykorzystania funkcji last_value do wskazywania najtańszego produktu w danej kategorii, musimy zmienić zakres tej funkcji:

```
-- wcześniej, zakres do aktualnego wiersza
last_value(productname) over (partition by categoryid
    order by unitprice desc) last

-- poprawiona wersja, zakres całego okna
last_value(productname) over (partition by categoryid
    order by unitprice desc rows between unbounded preceding and unbounded following) last
```

Rezultaty poprawionego zapytania:

	productid	productname	unitprice	categoryid	first	last
1	38	Côte de Blaye	263.5	1	Côte de Blaye	Guaraná Fantástica
2	43	Ipoh Coffee	46	1	Côte de Blaye	Guaraná Fantástica
3	2	Chang	19	1	Côte de Blaye	Guaraná Fantástica
4	1	Chai	18	1	Côte de Blaye	Guaraná Fantástica
5	76	Lakkalikööri	18	1	Côte de Blaye	Guaraná Fantástica
6	35	Steeleye Stout	18	1	Côte de Blaye	Guaraná Fantástica
7	39	Chartreuse verte	18	1	Côte de Blaye	Guaraná Fantástica
8	70	Outback Lager	15	1	Côte de Blaye	Guaraná Fantástica
9	34	Sasquatch Ale	14	1	Côte de Blaye	Guaraná Fantástica
10	67	Laughing Lumberjack Lager	14	1	Côte de Blaye	Guaraná Fantástica
11	75	Rhönbräu Klosterbier	7.75	1	Côte de Blaye	Guaraná Fantástica
12	24	Guaraná Fantástica	4.5	1	Côte de Blaye	Guaraná Fantástica
13	63	Vegie-spread	43.9	2	Vegie-spread	Aniseed Syrup

Wizualizacja 27: Zadanie 5 - poprawiony wynik first_value/last_value (PostgreSQL)

Zadanie 6

Baza: Northwind, tabele orders, order details

Napisz polecenie które wyświetla inf. o zamówieniach

Zbiór wynikowy powinien zawierać:

- Id klienta,
- nr zamówienia,
- datę zamówienia,
- wartość zamówienia (wraz z opłatą za przesyłkę),
- dane zamówienia klienta o najniższej wartości w danym miesiącu
 - nr zamówienia o najniższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia
- dane zamówienia klienta o najwyższej wartości w danym miesiącu
 - nr zamówienia o najwyższej wartości w danym miesiącu
 - datę tego zamówienia
 - wartość tego zamówienia

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite.

Wyniki:

-- PostgreSQL

```
with ordersummary as (select o.customerid,
                             o.orderid,
                             o.orderdate,
                             date_trunc('month', o.orderdate) as ordermonth,
                             sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                        from orders o
                         join orderdetails od on o.orderid = od.orderid
                        group by o.customerid, o.orderid, o.orderdate, o.freight)

select customerid,
       orderid,
       orderdate,
       ordertotal,
       first_value(orderid) over (partition by customerid, ordermonth order by ordertotal asc) as minorderid,
       first_value(orderdate) over (partition by customerid, ordermonth order by ordertotal asc) as minorderdate,
       first_value(ordertotal) over (partition by customerid, ordermonth order by ordertotal asc) as minordervalue,
       first_value(orderid) over (partition by customerid, ordermonth order by ordertotal desc) as maxorderid,
       first_value(orderdate) over (partition by customerid, ordermonth order by ordertotal desc) as maxorderdate,
       first_value(ordertotal) over (partition by customerid, ordermonth order by ordertotal desc) as maxordervalue
from ordersummary
order by customerid, orderdate;
```

	cu. ∇ ÷	or... ∇ ÷	orde... ∇ ÷	ordentot... ∇ ÷	minorderid ∇ ÷	minorderdate ∇ ÷	minordervalue ∇ ÷	maxorderid ∇ ÷	maxorderdate ∇ ÷	maxordervalue ∇ ÷		
1	ALFKI		10643	1997-08-25		843.959981918335		10643	1997-08-25		843.959981918335	
2	ALFKI		10692	1997-10-03		939.0200309753418		10692	1997-10-03		939.0200309753418	
3	ALFKI		10702	1997-10-13		353.9400005340576		10692	1997-10-03		939.0200309753418	
4	ALFKI		10835	1998-01-15		915.3299987018108		10835	1998-01-15		915.3299987018108	
5	ALFKI		10952	1998-03-16		511.6199948191643		10952	1998-03-16		511.6199948191643	
6	ALFKI		11011	1998-04-09		934.7099996432662		11011	1998-04-09		934.7099996432662	
7	ANATR		10308	1996-09-18		90.40999925136566		10308	1996-09-18		90.40999925136566	
8	ANATR		10625	1997-08-08		523.6500015258789		10625	1997-08-08		523.6500015258789	
9	ANATR		10759	1997-11-28		331.98999977111816		10759	1997-11-28		331.98999977111816	
10	ANATR		10926	1998-03-04		554.3199892044067		10926	1998-03-04		554.3199892044067	
11	ANTON		10365	1996-11-27		425.1999816894531		10365	1996-11-27		425.1999816894531	
12	ANTON		10507	1997-04-15		796.5124955102801		10507	1997-04-15		796.5124955102801	
13	ANTON		10535	1997-05-13		1956.48999369666		10535	1997-05-13		1956.48999369666	
14	ANTON		10573	1997-06-19		2166.8399772644043		10573	1997-06-19		2166.8399772644043	
15	ANTON		10677	1997-09-22		817.3949828332663		10677	1997-09-22		817.3949828332663	
16	ANTON		10682	1997-09-25		411.63000106811523		10677	1997-09-22		817.3949828332663	
17	ANTON		10856	1998-01-28		718.4300003051758		10856	1998-01-28		718.4300003051758	
18	AROUT		10355	1996-11-15		521.9500079154968		10355	1996-11-15		521.9500079154968	
< < 1-500 ∨ of 501+ > > :			996-12-16		933.2399978637695		933.2399978637695		10383	1996-12-16		933.2399978637695
20	AROUT		10653	1997-02-21		633.0500073606083		10653	1997-02-21		633.0500073606083	

Wizualizacja 28: Zadanie 6 - Wynik

Podejścia bez funkcji okna:

```
-- podzapytanie
with ordersummary as (select c.companyname,
                           o.customerid,
                           o.orderid,
                           o.orderdate,
                           date_trunc('month', o.orderdate) as ordermonth,
                           sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                           from orders o
                           join customers c on o.customerid = c.customerid
                           join orderdetails od on o.orderid = od.orderid
                           group by c.companyname, o.customerid, o.orderid, o.orderdate, o.freight)
select t1.customerid,
       t1.orderid,
       t1.orderdate,
       t1.ordertotal,
       (select t2.orderid
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal asc, t2.orderid asc
        limit 1) as minorderid,
       (select t2.orderdate
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal asc, t2.orderid asc
        limit 1) as minorderdate,
       (select t2.ordertotal
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal asc, t2.orderid asc
        limit 1) as minordervalue,
       (select t2.orderid
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal desc, t2.orderid desc
        limit 1) as maxorderid,
       (select t2.orderdate
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal desc, t2.orderid desc
        limit 1) as maxorderdate,
       (select t2.ordertotal
        from ordersummary t2
        where t2.customerid = t1.customerid
              and t2.ordermonth = t1.ordermonth
        order by t2.ordertotal desc, t2.orderid desc
        limit 1) as maxordervalue
from ordersummary t1
order by t1.customerid, t1.orderdate;
```

```
-- joiny
with ordersummary as (select c.companyname,
                           o.customerid,
                           o.orderid,
                           o.orderdate,
                           date_trunc('month', o.orderdate) as ordermonth,
                           sum(od.unitprice * od.quantity * (1 - od.discount)) + o.freight as ordertotal
                        from orders o
                           join customers c on o.customerid = c.customerid
                           join orderdetails od on o.orderid = od.orderid
                        group by c.companyname, o.customerid, o.orderid, o.orderdate, o.freight),
monthlyextremes as (select customerid,
                          ordermonth,
                          min(ordertotal) as minval,
                          max(ordertotal) as maxval
                    from ordersummary
                    group by customerid, ordermonth)

select os.customerid,
       os.orderid,
       os.orderdate,
       os.ordertotal,
       os_min.orderid as minorderid,
       os_min.orderdate as minorderdate,
       os_min.ordertotal as minordervalue,
       os_max.orderid as maxorderid,
       os_max.orderdate as maxorderdate,
       os_max.ordertotal as maxordervalue
from ordersummary os
     join monthlyextremes ex on os.customerid = ex.customerid and os.ordermonth = ex.ordermonth
     left join ordersummary os_min on os_min.customerid = ex.customerid
and os_min.ordermonth = ex.ordermonth
and os_min.ordertotal = ex.minval
     left join ordersummary os_max on os_max.customerid = ex.customerid
and os_max.ordermonth = ex.ordermonth
and os_max.ordertotal = ex.maxval
order by os.customerid, os.orderdate;
```

Porównanie wyników klazulą except w obie strony dało pusty zbiór wynikowy, co oznacza, że wyniki są takie same dla wszystkich trzech podejść.

Porównanie planów wykonania dla różnych podejść:

- funkcje okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
▼ ⌕ Select								Planning Time = 0....
▼ Unknown (Incremental Sort)		830	830	256.67	2.391	167.84	1.342	Parallel Aware = fa...
▼ Transformation (WindowAgg)		830	830	233.78	2.166	167.46	1.283	Parent Relationshi...
▼ Unknown (Incremental Sort)		830	830	213.03	1.831	167.46	1.281	Parent Relationshi...
▼ Transformation (WindowAgg)		830	830	190.13	1.624	167.31	1.256	Parent Relationshi...
▼ Sort		830	830	169.38	1.277	167.31	1.252	Parent Relationshi...
▼ 📄 Access (Subquery Scan)		830	830	127.07	0.905	104.24	0.735	Parent Relationshi...
▼ Aggregate		830	830	118.77	0.865	104.24	0.734	Strategy = Hashe...
▼ ⚡ Hash Join		2155	2155	71.92	0.478	32.67	0.133	Parent Relationshi...
📄 Full Scan (Seq Scan)	table: orderdetails;	2155	2155	33.55	0.08	0.0	0.004	Parent Relationshi...
▼ Transformation (Hash)		830	830	22.3	0.125	22.3	0.125	Parent Relationshi...
📄 Full Scan (Seq Scan)	table: orders;	830	830	22.3	0.068	0.0	0.003	Parent Relationshi...

Wizualizacja 29: Zadanie 6 - Plan wykonania (funkcje okna)

QUERY PLAN	
1	Incremental Sort (cost=167.84..256.67 rows=830 width=56) (actual time=1.296..2.325 rows=830 loops=1)
2	Sort Key: ordersummary.customerid, ordersummary.orderdate
3	Presorted Key: ordersummary.customerid
4	Full-sort Groups: 22 Sort Method: quicksort Average Memory: 29kB Peak Memory: 29kB
5	-> WindowAgg (cost=167.46..233.78 rows=830 width=56) (actual time=1.237..2.115 rows=830 loops=1)
6	-> Incremental Sort (cost=167.46..213.03 rows=830 width=42) (actual time=1.235..1.769 rows=830 loops=1)
7	Sort Key: ordersummary.customerid, ordersummary.ordermonth, ordersummary.ordertotal
8	Presorted Key: ordersummary.customerid, ordersummary.ordermonth
9	Full-sort Groups: 26 Sort Method: quicksort Average Memory: 27kB Peak Memory: 27kB
10	-> WindowAgg (cost=167.31..190.13 rows=830 width=42) (actual time=1.206..1.570 rows=830 loops=1)
11	-> Sort (cost=167.31..169.38 rows=830 width=28) (actual time=1.202..1.225 rows=830 loops=1)
12	Sort Key: ordersummary.customerid, ordersummary.ordermonth, ordersummary.ordertotal DESC
13	Sort Method: quicksort Memory: 70kB
14	-> Subquery Scan on ordersummary (cost=104.24..127.07 rows=830 width=28) (actual time=0.682..0.856 rows=...
15	-> HashAggregate (cost=104.24..118.77 rows=830 width=32) (actual time=0.682..0.816 rows=830 loops=...
16	Group Key: o.orderid
17	Batches: 1 Memory Usage: 105kB
18	-> Hash Join (cost=32.67..71.92 rows=2155 width=26) (actual time=0.121..0.443 rows=2155 loop=...
19	Hash Cond: (od.orderid = o.orderid)
20	-> Seq Scan on orderdetails od (cost=0.00..33.55 rows=2155 width=12) (actual time=0.00...
21	-> Hash (cost=22.30..22.30 rows=830 width=16) (actual time=0.115..0.115 rows=830 loops=...
22	Buckets: 1024 Batches: 1 Memory Usage: 47kB
23	-> Seq Scan on orders o (cost=0.00..22.30 rows=830 width=16) (actual time=0.002...
24	Planning Time: 0.173 ms
25	25 rows 2.393 ms

Wizualizacja 30: Zadanie 6 - Plan wykonania (funkcje okna, surowy)

- podzapytanie:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⌵ Select								Planning Time = 0.301;Trig...
⌵ Sort		2155	830	697122.82	488.713	697117.43	488.684	Parallel Aware = false;Asyn...
⌵ Aggregate		2155	830	157.31	332.251	119.6	332.116	Strategy = Hashed;Partial ...
⌵ ⌵ Hash Join		2155	2155	81.89	331.803	36.72	331.357	Parent Relationship = Out...
⌵ ⌵ Hash Join		2155	2155	71.92	0.382	32.67	0.124	Parent Relationship = Out...
⌵ ⌵ ⌵ Full Scan (Seq Scan)	table: orderdetails;	2155	2155	33.55	0.075	0.0	0.005	Parent Relationship = Out...
⌵ ⌵ Transformation (Hash)		830	830	22.3	0.109	22.3	0.108	Parent Relationship = Inne...
⌵ ⌵ ⌵ Full Scan (Seq Scan)	table: orders;	830	830	22.3	0.055	0.0	0.005	Parent Relationship = Out...
⌵ Transformation (Hash)		91	91	2.91	331.226	2.91	331.226	Parent Relationship = Inne...
⌵ ⌵ Full Scan (Seq Scan)	table: customers;	91	91	2.91	331.204	0.0	331.193	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	2155	830	696840.8	488.295	0.0	332.581	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...
⌵ Sort		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	1	2	53.88	0.031	0.0	0.013	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...
⌵ Sort		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	1	2	53.88	0.031	0.0	0.013	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...
⌵ Sort		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	1	2	53.88	0.03	0.0	0.013	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...
⌵ Sort		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	1	2	53.88	0.031	0.0	0.013	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...
⌵ Sort		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Out...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	1	2	53.88	0.031	0.0	0.013	Parent Relationship = Out...
⌵ Transformation (Limit)		1	1	53.89	0.031	53.88	0.031	Parent Relationship = Sub...

Operations Tree

Wizualizacja 31: Zadanie 6 - Plan wykonania (podzapytanie)

	⌵ QUERY PLAN ⌵
51	Rows Removed by Filter: 828
52	SubPlan 6
53	-> Limit (cost=53.88..53.89 rows=1 width=14) (actual time=0.032..0.032 rows=1 loops=830)
54	-> Sort (cost=53.88..53.89 rows=1 width=14) (actual time=0.032..0.032 rows=1 loops=830)
55	Sort Key: t2_4.ordertotal DESC, t2_4.orderid DESC
56	Sort Method: top-N heapsort Memory: 25kB
57	-> CTE Scan on ordersummary t2_4 (cost=0.00..53.88 rows=1 width=14) (actual time=0.013..0.013 rows=1 loops=1)
58	Filter: ((customerid = t1.customerid) AND (ordermonth = t1.ordermonth))
59	Rows Removed by Filter: 828
60	SubPlan 7
61	-> Limit (cost=53.88..53.89 rows=1 width=10) (actual time=0.032..0.032 rows=1 loops=830)
62	-> Sort (cost=53.88..53.89 rows=1 width=10) (actual time=0.032..0.032 rows=1 loops=830)
63	Sort Key: t2_5.ordertotal DESC, t2_5.orderid DESC
64	Sort Method: top-N heapsort Memory: 25kB
65	-> CTE Scan on ordersummary t2_5 (cost=0.00..53.88 rows=1 width=10) (actual time=0.013..0.013 rows=1 loops=1)
66	Filter: ((customerid = t1.customerid) AND (ordermonth = t1.ordermonth))
67	Rows Removed by Filter: 828
68	Planning Time: 0.267 ms
69	JIT:
70	Functions: 79
71	Options: Inlining true, Optimization true, Expressions true, Deforming true
72	Timing: Generation 1.964 ms, Inlining 8.413 ms, Optimization 189.395 ms, Emission 138.850 ms, Total 338.622 ms
73	Execution Time: 503.474 ms

Wizualizacja 32: Zadanie 6 - Plan wykonania (podzapytanie, surowy)

• joiny:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⌵ Select								Planning Time = 0.29;...
⌵ Sort		12	830	417.15	2.935	417.12	2.912	Parallel Aware = false;...
⌵ Aggregate		2155	830	157.31	1.235	119.6	1.043	Strategy = Hashed;Pa...
⌵ ⌵ Hash Join		2155	2155	81.89	0.717	36.72	0.141	Parent Relationship =...
⌵ ⌵ Hash Join		2155	2155	71.92	0.46	32.67	0.121	Parent Relationship =...
⌵ ⌵ Full Scan (Seq Scan)	table: orderdetails;	2155	2155	33.55	0.08	0.0	0.002	Parent Relationship =...
⌵ Transformation (Hash)		830	830	22.3	0.116	22.3	0.116	Parent Relationship =...
⌵ ⌵ Full Scan (Seq Scan)	table: orders;	830	830	22.3	0.064	0.0	0.002	Parent Relationship =...
⌵ Transformation (Hash)		91	91	2.91	0.018	2.91	0.018	Parent Relationship =...
⌵ ⌵ Full Scan (Seq Scan)	table: customers;	91	91	2.91	0.01	0.0	0.004	Parent Relationship =...
⌵ ⌵ Hash Join		12	830	259.59	2.548	192.24	2.352	Parent Relationship =...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	0.05	0.0	0.0	Parent Relationship =...
⌵ Transformation (Hash)		12	830	192.03	2.349	192.03	2.348	Parent Relationship =...
⌵ ⌵ Hash Join		12	830	192.03	2.255	124.67	2.054	Parent Relationship =...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	0.05	0.0	0.0	Parent Relationship =...
⌵ Transformation (Hash)		12	830	124.46	2.052	124.46	2.052	Parent Relationship =...
⌵ ⌵ Hash Join		12	830	124.46	1.929	70.05	1.715	Parent Relationship =...
⌵ ⌵ Access (CTE Scan)	table: ordersummary;	2155	830	43.1	1.112	0.0	1.045	Parent Relationship =...
⌵ Transformation (Hash)		216	636	66.81	0.666	66.81	0.666	Parent Relationship =...
⌵ Aggregate		216	636	66.81	0.579	64.65	0.51	Strategy = Hashed;Pa...
⌵ ⌵ Access (CTE Scan	table: ordersummary;	2155	830	43.1	0.361	0.0	0.0	Parent Relationship =...

Wizualizacja 33: Zadanie 6 - Plan wykonania (joiny)

⌵ QUERY PLAN ⌵	
15	-> Seq Scan on orders o (cost=0.00..22.30 rows=830 width=16) (actual time=0.002..0.064 rows=830 loops=1)
16	-> Hash (cost=2.91..2.91 rows=91 width=26) (actual time=0.017..0.018 rows=91 loops=1)
17	Buckets: 1024 Batches: 1 Memory Usage: 14kB
18	-> Seq Scan on customers c (cost=0.00..2.91 rows=91 width=26) (actual time=0.004..0.010 rows=91 loops=1)
19	-> Hash Right Join (cost=192.24..259.59 rows=12 width=74) (actual time=2.059..2.282 rows=830 loops=1)
20	Hash Cond: ((os_max.customerid = ordersummary.customerid) AND (os_max.ordermonth = ordersummary.ordermonth) AND (os_max.ordertotal = (max(ordersum...
21	-> CTE Scan on ordersummary os_max (cost=0.00..43.10 rows=2155 width=54) (actual time=0.000..0.058 rows=830 loops=1)
22	-> Hash (cost=192.03..192.03 rows=12 width=108) (actual time=2.056..2.057 rows=830 loops=1)
23	Buckets: 1024 Batches: 1 Memory Usage: 86kB
24	-> Hash Right Join (cost=124.67..192.03 rows=12 width=108) (actual time=1.768..1.963 rows=830 loops=1)
25	Hash Cond: ((os_min.customerid = ordersummary.customerid) AND (os_min.ordermonth = ordersummary.ordermonth) AND (os_min.ordertotal = (min(ordersummary.ordertotal))))
26	-> CTE Scan on ordersummary os_min (cost=0.00..43.10 rows=2155 width=54) (actual time=0.000..0.049 rows=830 loops=1)
27	-> Hash (cost=124.46..124.46 rows=12 width=102) (actual time=1.765..1.766 rows=830 loops=1)
28	Buckets: 1024 Batches: 1 Memory Usage: 80kB
29	-> Hash Join (cost=70.05..124.46 rows=12 width=102) (actual time=1.516..1.678 rows=830 loops=1)
30	Hash Cond: ((os.customerid = ordersummary.customerid) AND (os.ordermonth = ordersummary.ordermonth))
31	-> CTE Scan on ordersummary os (cost=0.00..43.10 rows=2155 width=54) (actual time=1.014..1.062 rows=830 loops=1)
32	-> Hash (cost=66.81..66.81 rows=216 width=56) (actual time=0.499..0.499 rows=636 loops=1)
33	Buckets: 1024 Batches: 1 Memory Usage: 48kB
34	-> HashAggregate (cost=64.65..66.81 rows=216 width=56) (actual time=0.405..0.445 rows=636 loops=1)
35	Group Key: ordersummary.customerid, ordersummary.ordermonth
36	Batches: 1 Memory Usage: 121kB
37	-> CTE Scan on ordersummary (cost=0.00..43.10 rows=2155 width=48) (actual time=0.000..0.285 rows=830 loops=1)
38	Planning Time: 0.286 ms
39	39 rows ⌵ ⋮
3	Time: 2.739 ms

Wizualizacja 34: Zadanie 6 - Plan wykonania (joiny, surowy)

Wnioski:

- funkcje okna znów okazały się najlepsze, zwłaszcza w porównaniu do podejścia z podzapytaniem, które jest bardzo kosztowne, co widać po czasie wykonania i kosztach
- zapytanie z joinami jest również kosztowne, ale znacznie mniej niż podejście z podzapytaniem, warto jednak zwrócić uwagę na `plan width (Estimated average width of rows output by this plan node (in bytes) .)`, który jest większy dla podejścia z joinami niż dla podejścia z funkcjami okna, co oznacza, że mimo podobnego czasu wykonania, podejście z joinami jest mniej wydajne pod względem wykorzystania pamięci niż podejście z funkcjami okna
- podejście z podzapytaniem znów wypadło najgorzej pod względem kosztu oraz czasu wykonania - dla każdego z 830 wierszy w tabeli wykonano 6 podzapytań, które filtowały 828 wierszy, warto zauważyć, że PostgreSQL nawet włączył JIT, aby przyspieszyć wykonanie, co zajęło ~340ms (dla porównania całość zapytania z funkcjami okna zajęło ~2.4ms!)

Zadanie 7

Baza: Northwind, tabela product_history

Napisz polecenie które pokaże wartość sprzedaży każdego produktu narastająco od początku każdego miesiąca. Użyj funkcji okna

Zbiór wynikowy powinien zawierać:

- id pozycji
- id produktu
- datę
- wartość sprzedaży produktu w danym dniu
- wartość sprzedaży produktu narastające od początku miesiąca

Spróbuj uzyskać ten sam wynik bez użycia funkcji okna, porównaj wyniki, czasy i plany zapytań (koszty).

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSQL, SQLite)

Wyniki:

Zapytanie realizujące opisane zadanie w Postgresie:

```
-- z funkcją okna
with t as (select od.orderid as id,
                 od.productid,
                 date_part('month', o.orderdate) as month,
                 o.orderdate as date,
                 od.unitprice * od.quantity * (1 - od.discount) as total,
                 sum(od.unitprice * od.quantity * (1 - od.discount)) over (
                     partition by od.productid,
                               date_part('year', o.orderdate),
                               date_part('month', o.orderdate)
                     order by od.productid, o.orderdate
                 ) as cum_total
            from orderdetails od
             join orders o on od.orderid = o.orderid)

select t.id,
       t.productid,
       t.month,
       t.date,
       t.total,
       t.cum_total

from t;

-- bez funkcji okna (inner-join)
with t as (select od.orderid as id,
                 od.productid,
                 date_part('month', o.orderdate) as month,
                 o.orderdate as date,
                 od.unitprice * od.quantity * (1 - od.discount) as total
            from orderdetails od
             join orders o on od.orderid = o.orderid)

select t.id,
       t.productid,
       t.month,
       t.date,
       t.total,
       sum(t2.total) as cum_total

from t
     left join t t2
         on t2.productid = t.productid
        and date_part('year', t2.date) = date_part('year', t.date)
        and date_part('month', t2.date) = date_part('month', t.date)
        and (
            t2.date < t.date
            or (t2.date = t.date and t2.id <= t.id)
        )

group by t.id,
         t.productid,
         t.month,
         t.date,
         t.total

order by t.productid,
         t.date;
```


Rezultaty zapytania:

- z funkcją okna:

	id	productid	month	date	total	cum_total
1	10285	1	8	1996-08-20	595.2299861669541	595.2299861669541
2	10294	1	8	1996-08-30	406.45998764038086	1001.6899738073349
3	10317	1	9	1996-09-30	300.68999195098877	300.68999195098877
4	10348	1	11	1996-11-07	184.37999382019046	184.37999382019046
5	10354	1	11	1996-11-14	226.59999465942383	410.9799884796143
6	10370	1	12	1996-12-03	184.76999380588535	184.76999380588535
7	10406	1	1	1997-01-07	252.03999710083008	252.03999710083008
8	10413	1	1	1997-01-14	441.25999450683594	693.299991607666
9	10477	1	3	1997-03-17	229.01999473571777	229.01999473571777
10	10522	1	4	1997-04-30	621.3299996852875	621.3299996852875
11	10526	1	5	1997-05-05	180.989999294281	180.989999294281
12	10576	1	6	1997-06-23	198.55999946594238	198.55999946594238
13	10590	1	7	1997-07-07	404.7700004577637	404.7700004577637
14	10609	1	7	1997-07-24	55.85000002384186	460.62000048160553
15	10611	1	7	1997-07-25	188.6500015258789	649.2700020074844
16	10628	1	8	1997-08-12	480.36000061035156	480.36000061035156
17	10646	1	8	1997-08-27	344.8300018310547	825.1900024414062
18	10689	1	10	1997-10-01	485.92000007629395	485.92000007629395

Wizualizacja 35: Zadanie 7 - wynik sumy narastającej z funkcją okna (PostgreSQL)

• z inner-joinem:

	id	productid	month	date	total	cum_total
1	10285	1	8	1996-08-20	595.2299861669541	595.2299861669541
2	10294	1	8	1996-08-30	406.45998764038086	1001.6899738073349
3	10317	1	9	1996-09-30	300.68999195098877	300.68999195098877
4	10348	1	11	1996-11-07	184.37999382019046	184.37999382019046
5	10354	1	11	1996-11-14	226.59999465942383	410.9799884796143
6	10370	1	12	1996-12-03	184.76999380588535	184.76999380588535
7	10406	1	1	1997-01-07	252.03999710083008	252.03999710083008
8	10413	1	1	1997-01-14	441.25999450683594	693.299991607666
9	10477	1	3	1997-03-17	229.01999473571777	229.01999473571777
10	10522	1	4	1997-04-30	621.3299996852875	621.3299996852875
11	10526	1	5	1997-05-05	180.989999294281	180.989999294281
12	10576	1	6	1997-06-23	198.55999946594238	198.55999946594238
13	10590	1	7	1997-07-07	404.7700004577637	404.7700004577637
14	10609	1	7	1997-07-24	55.85000002384186	460.62000048160553
15	10611	1	7	1997-07-25	188.6500015258789	649.2700020074844
16	10628	1	8	1997-08-12	480.36000061035156	480.36000061035156
17	10646	1	8	1997-08-27	344.8300018310547	825.1900024414062
18	10689	1	10	1997-10-01	485.92000007629395	485.92000007629395

Wizualizacja 36: Zadanie 7 - wynik sumy narastającej z inner-join (PostgreSQL)

Jak widać na załączonych zrzutach ekranu, wartości te są kumulowane w ramach danego miesiąca. W celu upewnienia się, że wyniki są identyczne w przypadku obu zapytań, ponownie korzystamy z dwukierunkowego except, który zwrócił pusty zbiór wynikowy, co oznacza, że wyniki są takie same dla obu podejść.

Porównanie wydajności i planów zapytań - Postgres:

- zapytanie z funkcją okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning = {"Shar...
⌵ 🗒 Access (Subquery Scan)		2155	2155.0	369.02	3.036	212.78	2.091	Parallel Aware = f...
⌵ Transformation (WindowAgg		2155	2155.0	347.47	2.906	212.78	2.09	Parent Relationsh...
⌵ Sort		2155	2155.0	218.17	2.192	212.78	2.053	Parent Relationsh...
⌵ ⚡ Hash Join		2155	2155.0	93.47	1.086	32.68	0.434	Parent Relationsh...
🗒 Full Scan (Seq S table: orderdetails;		2155	2155.0	33.55	0.214	0.0	0.038	Parent Relationsh...
⌵ Transformation (H		830	830.0	22.3	0.331	22.3	0.327	Parent Relationsh...
🗒 Full Scan (Se table: orders;		830	830.0	22.3	0.202	0.0	0.017	Parent Relationsh...

Wizualizacja 37: Zadanie 7 - plan zapytania z funkcją okna (PostgreSQL)

- zapytanie z inner-joinem:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
⌵ ⇐ Select								Planning = {"Shared ...
⌵ Sort		1246	2155.0	675.76	7.112	672.64	7.043	Parallel Aware = fals...
⌵ ⚡ Hash Join		2155	2155.0	109.63	1.321	32.68	0.712	Parent Relationship ...
🗒 Full Scan (Seq Scan) table: orderdetails;		2155	2155.0	33.55	0.284	0.0	0.105	Parent Relationship ...
⌵ Transformation (Hash)		830	830.0	22.3	0.521	22.3	0.52	Parent Relationship ...
🗒 Full Scan (Seq Scan table: orders;		830	830.0	22.3	0.322	0.0	0.03	Parent Relationship ...
⌵ Aggregate		1246	2155.0	498.95	6.291	486.49	6.092	Strategy = Hashed; ...
⌵ ⚡ Merge Join		2155	3851.0	454.17	5.319	324.83	4.005	Parent Relationship ...
⌵ Sort		2155	2155.0	167.8	3.157	162.42	3.052	Parent Relationship ...
🗒 Access (CTE Sc table: t;		2155	2155.0	43.1	2.141	0.0	0.717	Parent Relationship ...
⌵ Sort		2155	5546.0	167.8	1.216	162.42	0.94	Parent Relationship ...
🗒 Access (CTE Sc table: t;		2155	2155.0	43.1	0.322	0.0	0.005	Parent Relationship ...

Wizualizacja 38: Zadanie 7 - plan zapytania z inner-join (PostgreSQL)

Wnioski:

- zapytanie z funkcją okna charakteryzuje się około 2 razy niższym kosztem zapytania
- ze względu na mały rozmiar danych (tabele orders i ordershistory), zapytania są generalnie bardzo szybkie (~30ms), więc nie ma sensu porównywać bezpośrednio czasu zapytań

Zapytania dla MSSQL oraz SQLite są identyczne, z dokładnością do funkcji ekstrahującej miesiąc oraz rok z daty:

```
-- postgres
date_part('year', o.orderdate) as year
date_part('month', o.orderdate) as month

-- mssql
year(o.orderdate) as year
month(o.orderdate) as month

-- sqlite
strftime('%Y', o.orderdate) as year
strftime('%m', o.orderdate) as month
```

Porównanie wydajności i planów zapytań - MSSQL:

- z funkcją okna:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
⌵ ⇐ Select		2155		0.103945		CardinalityEstimation...
⌵ ▢ Value (Compute Scalar)		2155		0.103945		AvgRowSize = 34; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.103729		AvgRowSize = 50; Es...
⌵ Aggregate (Aggregate - Stream Aggregate)		2155	2155	0.103514	40.0	AvgRowSize = 75; Es...
⌵ ▢ Temporary (Window Spool)		4310	4310	0.10216	36.0	AvgRowSize = 75; Es...
⌵ Transformation (Segment)		2155	2155	0.0968824	11.0	AvgRowSize = 75; Es...
⌵ Transformation (Segment)		2155	2155	0.0968824	10.0	AvgRowSize = 67; Es...
⌵ Sort		2155	2155	0.096747	10.0	AvgRowSize = 59; Es...
⌵ ⌵⇐ Merge Join (Inner Join)		2155	2155	0.0481589	3.0	AvgRowSize = 59; Es...
⌵ ▢ Value (Compute Scalar)		830		0.021315		AvgRowSize = 34; Es...
🔑 Full Index Scan (table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	0.0	AvgRowSize = 22; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.0147569		AvgRowSize = 35; Es...
🔑 Full Index Scan (table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	1.0	AvgRowSize = 31; Es...

Wizualizacja 39: Zadanie 7 - plan zapytania z funkcją okna (MSSQL)

- z inner-join:

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
⌵ ⇐ Select		2155		0.363612		CardinalityEstimation...
⌵ Sort		2155	2155	0.363612	34.0	AvgRowSize = 34; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.315023		AvgRowSize = 34; Es...
⌵ Aggregate (Aggregate - Stream Aggregate)		2155	2155	0.315023	32.0	AvgRowSize = 34; Es...
⌵ Sort		2155	3851	0.312653	31.0	AvgRowSize = 30; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.264065		AvgRowSize = 30; Es...
⌵ ⌵⇐ Hash Join (Right Outer Join - Hash)		2155	3851	0.263849	28.0	AvgRowSize = 34; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.0798844		AvgRowSize = 30; Es...
⌵ ⌵⇐ Hash Join (Inner Join - Hash)		2155	2155	0.0796689	15.0	AvgRowSize = 34; Es...
⌵ ▢ Value (Compute Scalar)		830		0.021315		AvgRowSize = 26; Es...
🔑 Full Index Scan (table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	2.0	AvgRowSize = 22; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.0147569		AvgRowSize = 19; Es...
🔑 Full Index Scan (table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	9.0	AvgRowSize = 31; Es...
⌵ ⌵⇐ Hash Join (Inner Join - Hash)		2155	2155	0.0796689	2.0	AvgRowSize = 34; Es...
⌵ ▢ Value (Compute Scalar)		830		0.021315		AvgRowSize = 26; Es...
🔑 Full Index Scan (table: [dbo].[orders]; index: [pk_orders];		830	830	0.021232	0.0	AvgRowSize = 22; Es...
⌵ ▢ Value (Compute Scalar)		2155		0.0147569		AvgRowSize = 19; Es...
🔑 Full Index Scan (table: [dbo].[orderdetails]; index: [pk_order_details];		2155	2155	0.0145414	1.0	AvgRowSize = 31; Es...

Wizualizacja 40: Zadanie 7 - plan zapytania z inner-join (MSSQL)

Wnioski:

- koszt całkowity dla funkcji okna jest ~3 krotnie niższy, natomiast czas wykonania jest bardzo zbliżony (ze względu na małą ilość danych)

Porównanie wydajności i planów zapytań - SQLite:

- z funkcją okna:

Operation	Params	Raw Desc
⌵ ⇐ Select		
⌵ Operation (CO-ROUTINE)		CO-ROUTINE t
⌵ Operation (CO-ROUTINE)		CO-ROUTINE (subquery-3)
🔑 Index Scan	table: od; index: ProductsOrder_Details;	SCAN od USING INDEX ProductsOrder_Details
🔑 Index Scan	table: o;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
Transformation (LAST 4 TERMS OF ORDER BY)		USE TEMP B-TREE FOR LAST 4 TERMS OF ORDER BY
📊 Full Scan	table: (subquery-3);	SCAN (subquery-3)
📊 Full Scan	table: t;	SCAN t

Wizualizacja 41: Zadanie 7 - plan zapytania z funkcją okna (SQLite)

- z inner-join:

Operation	Params	Raw Desc
⌵ ⇐ Select		
⌵ Operation (MATERIALIZE)		MATERIALIZE t
📊 Full Scan	table: od;	SCAN od
🔑 Index Scan	table: o;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
🔑 Index Scan	table: od; index: sqlite_autoindex_Order;	SCAN od USING INDEX sqlite_autoindex_Order Details_1
🔑 Index Scan	table: o;	SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
Unknown		BLOOM FILTER ON t2 (productid=? AND month=?)
🔑 Index Scan	table: t2; index: (productid=?;	SEARCH t2 USING AUTOMATIC COVERING INDEX (productid=? ..
Order By		USE TEMP B-TREE FOR ORDER BY

Wizualizacja 42: Zadanie 7 - plan zapytania z inner-join (SQLite)

Wnioski:

- czas wykonania zapytań jest bardzo zbliżony (~400ms) zarówno dla funkcji okna i inner-joina, jest to około 20-krotnie dłużej niż to samo zapytanie w Postgres i MSSQL

Zadanie 8

Wykonaj kilka “własnych” przykładowych analiz.

Czy są jeszcze jakieś ciekawe/przydatne funkcje okna (z których nie korzystałeś w ćwiczeniu)? Spróbuj ich użyć w zaprezentowanych przykładach.

Do analizy użyj wybranego systemu/bazy danych - wybierz MS SQLserver, Postgres lub SQLite.

1. ntile (num_buckets integer) → integer

Funkcja ntile dzieli uporządkowany zbiór wynikowy na num_buckets grup (wierszy) i przypisuje każdemu wierszowi numer grupy, do której należy. Grupy są numerowane od 1 do num_buckets.
Np. Podziel klientów na 4 grupy (kwartyle) na podstawie ich całkowitych wydatków w 1997 roku.

Wyniki:

```
-- PostgreSQL
with customerspending as (select c.companyname,
                                c.customerid,
                                sum(od.unitprice * od.quantity * (1 - od.discount)) as totalspent
                            from orders o
                                join customers c on o.customerid = c.customerid
                                join orderdetails od on o.orderid = od.orderid
                            where o.orderdate >= '1997-01-01'
                                and o.orderdate <= '1997-12-31'
                            group by c.companyname, c.customerid)
select companyname,
       totalspent,
       ntile(4) over (order by totalspent desc) as customertier
from customerspending
order by customertier, totalspent desc;
```

Początek wyniku:

	companyname	totalspent	customertier
1	QUICK-Stop	61109.910123648275	1
2	Save-a-lot Markets	57713.57465326577	1
3	Ernst Handel	48096.26317733166	1
4	Mère Paillarde	23332.310070187	1
5	Hungry Owl All-Night Grocers	20454.404859158098	1
6	Rattlesnake Canyon Grocery	19383.750215929	1
7	Simons bistro	16232.412604600191	1
8	Berglunds snabbköp	13849.014920111225	1
9	HILARION-Abastos	13482.743995961011	1
10	Folk och fä HB	13314.669975699782	1
11	Lehmanns Marktstand	13076.127438613139	1
12	Wartian Herkku	12262.942562392802	1
13	Richter Supermarkt	11864.42400320208	1
14	Frankenversand	11829.781985437081	1
15	Folies gourmandes	11666.900089263916	1
16	Bon app'	11208.362467606552	1
17	Queen Cozinha	10132.76750496205	1
18	Königlich Essen	9664.209983877987	1
19	Piccolo und mehr	9305.579979383649	1
20	White Clover Markets	9146.504936572612	1
21	Seven Seas Imports	9021.244968587085	1
22	Vaffeljernet	8960.12485077791	1
23	Food Market	8565.31499237217	2

Wizualizacja 43: Zadanie 8 - Wynik (początek)

Koniec wyniku:

	companyname	totalspent	customertier
66	Wolski Zajazd	1207.8499994277954	4
67	Wilman Kala	1174.3500022888184	4
68	Rancho grande	1149.3999938964844	4
69	Comércio Mineiro	1127.9999828338623	4
70	Blauer See Delikatessen	1079.799991607666	4
71	France restauration	920.0999908447266	4
72	Ana Trujillo Emparedados y helados	799.75	4
73	Consolidated Holdings	787.5999774932861	4
74	Santé Gourmet	699.9999961853027	4
75	North/South	604	4
76	Galería del gastrónomo	493.20000076293945	4
77	Du monde entier	487	4
78	Océano Atlántico Ltda.	429.2000045776367	4
79	Drachenblut Delikatessen	420	4
80	GROSELLA-Restaurante	387.5	4
81	Vins et alcools Chevalier	379.7999954223633	4
82	Lazy K Kountry Store	356.99999809265137	4
83	Laughing Bacchus Wine Cellars	335.49999237060547	4
84	Franchi S.p.A.	249.6999979019165	4
85	Cactus Comidas para llevar	237.99999618530273	4
86	Spécialités du monde	52.35000228881836	4

Wizualizacja 44: Zadanie 8 - Wynik (koniec)

2. percent_rank () → double precision,cume_dist () → double precision

Funkcja percent_rank oblicza procentową pozycję wiersza w zbiorze wynikowym, natomiast cume_dist oblicza skumulowany procent wierszy, które mają wartość mniejszą lub równą wartości w bieżącym wierszu.

Np. Utwórz zestawienie sprzedaży dla produktu Chef Anton's Cajun Seasoning i oblicz percentyl oraz skumulowaną dystrybucję wartości sprzedaży tego produktu. Podaj nazwę klienta, nr zamówienia, wartość sprzedaży, percentyl oraz skumulowaną dystrybucję wartości sprzedaży dla tego produktu.

```
-- PostgreSQL
with productsales as (select c.companyname,
                           p.productname,
                           o.orderid,
                           (od.unitprice * od.quantity) as salevalue
                           from orderdetails od
                           join products p on od.productid = p.productid
                           join orders o on od.orderid = o.orderid
                           join customers c on o.customerid = c.customerid)

select companyname,
       orderid,
       productname,
       salevalue,
       round(percent_rank() over (
         partition by productname
         order by salevalue
      )::numeric, 3) as percentilerank,
       round(cume_dist() over (
         partition by productname
         order by salevalue
      )::numeric, 3) as cumulativedistribution
from productsales
where productname = 'Chef Anton's Cajun Seasoning'
order by salevalue desc;
```

	companyname	orderid	productname	salevalue	percentilerank	cumulativedistribution
1	Folk och fä HB	10533	Chef Anton's Cajun Season...	1100	0.895	1
2	Bon app'	10511	Chef Anton's Cajun Season...	1100	0.895	1
3	QUICK-Stop	10527	Chef Anton's Cajun Season...	1100	0.895	1
4	Queen Cozinha	10913	Chef Anton's Cajun Season...	660	0.842	0.85
5	White Clover Markets	10344	Chef Anton's Cajun Season...	616.0000133514404	0.789	0.8
6	Eastern Connection	10726	Chef Anton's Cajun Season...	550	0.632	0.75
7	Rattlesnake Canyon Grocery	11000	Chef Anton's Cajun Season...	550	0.632	0.75
8	Wartian Herkku	10636	Chef Anton's Cajun Season...	550	0.632	0.75
9	Suprêmes délices	10846	Chef Anton's Cajun Season...	462	0.579	0.6
10	Tradição Hipermercados	10606	Chef Anton's Cajun Season...	440	0.526	0.55
11	Bólido Comidas preparadas	10326	Chef Anton's Cajun Season...	422.40000915527344	0.474	0.5
12	Hungry Owl All-Night Grocers	10309	Chef Anton's Cajun Season...	352.00000762939453	0.421	0.45
13	Princesa Isabel Vinhos	10336	Chef Anton's Cajun Season...	316.8000068664551	0.368	0.4
14	Furia Bacalhau e Frutos do ...	10464	Chef Anton's Cajun Season...	281.6000061035156	0.316	0.35
15	Berglunds snabbköp	10654	Chef Anton's Cajun Season...	264	0.263	0.3
16	Magazzini Alimentari Riuniti	10635	Chef Anton's Cajun Season...	220	0.211	0.25
17	Mère Paillarde	10339	Chef Anton's Cajun Season...	176.00000381469727	0.158	0.2
18	Queen Cozinha	10704	Chef Anton's Cajun Season...	132	0.105	0.15
19	Magazzini Alimentari Riuniti	10950	Chef Anton's Cajun Season...	110	0.053	0.1
20	Rattlesnake Canyon Grocery	11077	Chef Anton's Cajun Season...	22	0	0.05

Wizualizacja 45: Zadanie 8 - Wynik (percentyl i skumulowana dystrybucja)

- Przy tym zestawieniu ładnie widać podział na klientów detalicznych i liderów sprzedaży.

zadanie	pkt
1	1
2	2
3	1
4	1
5	1
6	1
7	2
8	2
razem	11